SOFTWARE DEVELOPMENT LIFE CYCLE

Complete Notes • Beginner to Advanced

Understand every SDLC phase through one continuous real-world-style example: SmartAttend — a College Attendance Management System.

Includes

7 SDLC phases • Feasibility study • SRS • HLD/LLD • Coding • Testing • Deployment • Maintenance • 9 SDLC models • Agile/Scrum/DevOps • Interview Q&A • Practice MCQs

A study guide for college exams, technical assessments, interviews, and practical software development.

How to Use These Notes

Read the SmartAttend case study as one continuous story. Each phase answers a different question: why build it, what it must do, how to design it, how to code it, how to verify it, how to release it, and how to maintain it.

- **For exams:** Memorize each phase, its purpose, activities, and deliverables.
- **For interviews:** Practise explaining each concept using the SmartAttend example.
- **For practical work:** Use the requirement, design, testing, and deployment checklists as a starting point.

Contents

- 1. Introduction to SDLC
- 2. The Seven Phases of SDLC
- 3. Planning and Feasibility Study
- 4. Requirements Analysis and SRS
- 5. System Design: HLD, LLD, Database, APIs
- 6. Implementation and Coding
- 7. Software Testing
- 8. Deployment
- 9. Maintenance
- 10. Complete SmartAttend Walkthrough
- 11. SDLC Models and Comparison
- 12. Agile, Scrum, and DevOps
- 13. Related Concepts and Documentation
- 14. Common Problems and Best Practices
- 15. Interview Questions and Practice MCQs

Case study used throughout

SmartAttend is a proposed web-based college attendance system. Students can view attendance, teachers can record it, and administrators can manage users and reports. All budgets, schedules, and sample requirements in these notes are illustrative.

1. Introduction to SDLC

1.1 What is software?

Software is a collection of programs, procedures, and related data that instructs a computer to perform specific tasks. Examples include WhatsApp, Google Chrome, banking applications, online shopping platforms, and college management systems.

1.2 Definition of SDLC

Software Development Life Cycle (SDLC) is a systematic process used to plan, design, develop, test, deploy, and maintain software applications. It defines activities performed at each stage to deliver reliable, secure, and maintainable software.

Why do we need SDLC?

- Prevents developers from coding before understanding the requirements.
- Helps manage scope, schedule, budget, and resources.
- Encourages suitable architecture and security decisions.
- Finds defects before they reach users.
- Makes delivery, documentation, and maintenance more predictable.

SmartAttend problem

A college currently records attendance in paper registers. Teachers spend time recording and calculating attendance, students cannot easily check their percentages, and administrators may make reporting errors. The proposed solution is a web application for recording attendance, calculating percentages, and generating reports.

Common seven-phase SDLC

1. PLANNING	2. REQUIREMENTS	3. DESIGN
Problem, scope, feasibility	User needs and SRS	Architecture and blueprint
4. IMPLEMENTATION	**5. TESTING**	**6. DEPLOYMENT**
Write and integrate code	Verify quality	Release to users
7. MAINTENANCE	**Feedback / iteration**	**Lifecycle continues**
Fix, adapt, improve	Revisit earlier work when needed	Not simply a one-way process

Different textbooks and organizations may combine or separate some phases. Sequential models move largely phase by phase; iterative and Agile approaches revisit earlier work as requirements and feedback evolve.

2. Phase 1 — Planning and Feasibility Study

Planning identifies the problem, objectives, stakeholders, scope, resources, schedule, budget, and major risks. It establishes whether the project is worth pursuing and how it may be delivered.

Activities

- Identify the problem and proposed solution.
- Define measurable project objectives and scope.
- Identify stakeholders and required resources.
- Estimate time, cost, staffing, and infrastructure.
- Conduct feasibility analysis and identify risks.
- Create the initial project plan and milestones.

SmartAttend objectives

- Reduce the time required to record attendance.
- Calculate attendance percentages automatically.
- Allow students to view attendance online.
- Generate accurate reports.

• Provide role-based access for students, teachers, and administrators.

Scope example

Included in first release	Excluded from first release
Login; teacher attendance marking; student attendance view; percentage calculation; reports; admin user management	Face recognition; biometric devices; AI absence prediction; payroll; online fee payments; native mobile app

Stakeholders

Stakeholder	Interest / responsibility
College management	Approves project and budget
Students	View their attendance
Teachers	Record attendance
Administrators	Manage users, subjects, and reports
Project manager	Coordinates delivery
Developers and testers	Build and verify the application

Feasibility study — TELOS

Type	Question	SmartAttend example
Technical	Can we build it?	The team has skills in React, Spring Boot, PostgreSQL, and hosting.
Economic	Is it financially worthwhile?	Compare development and operating costs with time saved and accuracy benefits.
Legal	Can it comply with applicable rules?	Protect student information, apply access controls, and use licensed software.
Operational	Will people use it effectively?	Teachers can mark attendance easily; students can understand their dashboards.
Schedule	Can it be delivered on time?	Check whether the team can meet a 12-week target.

Illustrative cost and payback

Expense	Illustrative estimate
Development	■40,000
Hosting for one year	■6,000
Testing and deployment	■4,000
Total	■50,000

If estimated annual savings are ■24,000, a simple payback period is ■50,000 ÷ ■24,000 ≈ 2.08 years. These figures are examples, not actual quotations. A real analysis should include recurring costs, maintenance, and non-financial benefits.

Risk examples

Risk	Possible response
Requirements change	Use an approval and change-control process.
Data loss	Back up data and test recovery.
Unauthorized access	Use authentication and server-side authorization.
Development delays	Track milestones, dependencies, and risks.
Incorrect records	Validate input and maintain audit logs.

Deliverables: project proposal, problem statement, scope, feasibility study, budget estimate, schedule, risk register, and initial project plan.

Interview: Why is planning important?

Planning establishes objectives, scope, feasibility, resources, schedule, budget, and risks. It determines whether a project is worth pursuing and provides a foundation for the remaining phases.

3. Phase 2 — Requirements Analysis

Requirements analysis collects, examines, documents, and validates stakeholder needs. Planning identifies the problem; requirements analysis specifies exactly what the software must do and the constraints under which it must operate.

Requirement-gathering techniques

Technique	SmartAttend example
Interviews	Ask teachers how they mark attendance, what reports they need, and whether records can be edited.
Questionnaires	Ask students how they want to view attendance.
Observation	Watch a teacher use a paper register and identify manual steps.
Document analysis	Review attendance registers, reports, and college policies.
Prototyping	Show an early login page and dashboard to collect feedback.

Functional requirements — what the system does

ID	Requirement
FR-01	Authorized users can log in.
FR-02	Teachers can view assigned classes.
FR-03	Teachers can mark attendance.
FR-04	Students can view their attendance.
FR-05	The system calculates attendance percentages.
FR-06	Administrators can manage student accounts.
FR-07	The system generates attendance reports.
FR-08	Unauthorized users cannot modify attendance.

Non-functional requirements — how well the system operates

Category	Example requirement
Performance	Common dashboard requests meet an agreed response-time target.
Security	Passwords use an appropriate password-hashing algorithm.
Availability	Service meets its agreed availability target.
Usability	Teachers can mark attendance without extensive training.
Reliability	Records are not silently lost during normal operation.
Scalability	System supports the expected user population.
Maintainability	Code is modular and documented.
Compatibility	Application works in supported browsers.

Remember: Functional = what the system does. Non-functional = quality attributes and constraints. Example functional requirement: a teacher can save attendance. Example non-functional requirement: saving attendance should complete within two seconds under a defined load.

User requirements vs. system requirements

User requirement: “A teacher must be able to mark attendance for a class.”
System requirement: “The system shall display enrolled students for the selected class and date, accept a status for each student, validate the submission, and persist the records.”

Use case: Mark Attendance

Field	Description
Actor	Teacher
Precondition	Teacher is authenticated and permitted to access the class.
Main flow	Log in → select class and subject → view enrolled students → mark status → submit → system validates → saves records → confirms success.
Alternative flow	If data is invalid or incomplete, show a validation message and allow correction.
Postcondition	Records are saved, or the system clearly reports failure.

Software Requirements Specification (SRS)

An SRS is a structured document that specifies the requirements of the software. A typical SRS includes introduction, purpose, scope, overall description, user classes, functional and non-functional requirements, interfaces, data requirements, constraints, assumptions, and acceptance criteria.

Example SRS requirement — attendance calculation

Requirement ID: FR-05. The system shall calculate attendance percentage using classes attended and total classes conducted for a selected subject.

```
Attendance Percentage = (Classes Attended / Total Classes Conducted) × 100
Example: (36 / 40) × 100 = 90%
```

The actual specification must explain how cancelled classes, excused absences, and institutional exceptions are handled. In this example, if no classes have been conducted, the application displays 0%; another design might display “Not available.”

Requirements validation checks that requirements are correct, complete, consistent, feasible, and testable. For example, resolve any conflict between “students can view attendance” and “only teachers can view attendance” by clarifying the intended permissions.

Deliverables: approved SRS, use cases, user stories where applicable, traceability information, acceptance criteria, and validated requirements.

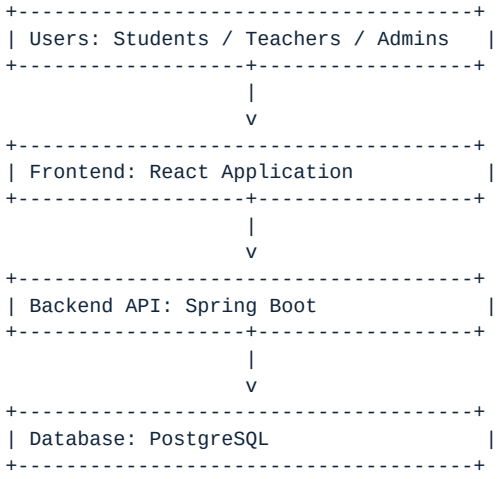
Interview: Requirement gathering collects information; requirements analysis organizes it and resolves ambiguity, conflicts, feasibility concerns, and missing details.

4. Phase 3 — System Design

System design converts approved requirements into a technical blueprint. Requirements describe what the system must do; design explains how the system will be structured to meet them.

High-Level Design (HLD)

HLD describes the overall architecture and major components.



Low-Level Design (LLD)

LLD describes internal component details such as classes, methods, database columns, constraints, API formats, algorithms, validation, and error handling.

```
public class AttendanceService {
    markAttendance(...);
    calculatePercentage(...);
    getStudentAttendance(...);
}
```

The method names above are illustrative; the exact design depends on the implementation.

Database design

Table	Example columns	Purpose
Students	student_id, name, email, password_hash	Student account and profile
Teachers	teacher_id, name, email	Teacher account
Subjects	subject_id, subject_name, teacher_id	Subject and assigned teacher
Attendance	attendance_id, student_id, subject_id, attendance_date, status	Attendance record

The database should define foreign keys, validation constraints, and uniqueness rules. For example, it may prevent duplicate attendance entries for the same student, subject, and class session.

UI design

```
-----------------------------------------
            SMARTATTEND
-----------------------------------------
Welcome, Ravi!

Subject              Attendance
-----------------------------------------
Mathematics           90%
Database Systems      85%
Operating Systems     76%
Computer Networks     92%
-----------------------------------------
Overall Attendance: 86%

[View Detailed Report]
-----------------------------------------
```

API design

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate a user
GET	/api/teacher/classes	Retrieve assigned classes
POST	/api/attendance	Submit attendance
GET	/api/students/me/attendance	Retrieve a student's attendance
GET	/api/reports/attendance	Generate an authorized report

The backend must enforce permissions independently of the frontend. A student must not be able to modify attendance simply by sending a request directly to an API endpoint.

Security design checklist

- Authentication and role-based authorization.
- Password hashing and input validation.
- Protection against common web vulnerabilities.
- HTTPS and secure secret management.
- Audit logging for important attendance changes.
- Backup and recovery procedures.

Deliverables: system architecture, HLD/LLD, database schema, UI wireframes, API specifications, security design, data-flow diagrams, component and class diagrams.

Interview: HLD describes the overall architecture and interactions; LLD describes internal component implementation details.

5. Phase 4 — Implementation (Coding)

Implementation translates the approved design into executable software. Developers write source code, create database objects, implement APIs, and integrate components.

Example technology stack

Layer / purpose	Example technology
Frontend	React
Backend	Java and Spring Boot
Database	PostgreSQL
API communication	REST
Version control	Git
Build	Maven
Testing	JUnit and integration-testing tools

Coding example: attendance calculator

```
public class AttendanceCalculator {

    public double calculatePercentage(
            int attendedClasses,
            int totalClasses) {

        if (attendedClasses < 0 || totalClasses < 0) {
            throw new IllegalArgumentException(
                    "Class counts cannot be negative");
        }

        if (attendedClasses > totalClasses) {
            throw new IllegalArgumentException(
                    "Attended classes cannot exceed total");
        }

        if (totalClasses == 0) {
            return 0.0;
        }

        return (attendedClasses * 100.0) / totalClasses;
    }
}
```

Input: 36 attended classes and 40 total classes. Calculation: 36 × 100 ÷ 40 = 90. Output: 90.0.

The method rejects negative counts and attended counts greater than the total. Returning zero when no classes have been conducted is an example policy; production requirements may choose a different representation.

Coding best practices

- Use meaningful names and small, focused methods.
- Build reusable components and validate input.
- Handle exceptions appropriately and follow secure coding practices.
- Use consistent formatting and document non-obvious logic.
- Write unit tests and review code.

Version control example

```
git init
git add .
git commit -m "Initial SmartAttend implementation
```

Version control records source-code changes and supports collaboration. Developers can work on separate branches and merge changes after review.

Integration example

The login page communicates with the authentication API; the teacher dashboard retrieves assigned classes; the attendance form submits records; the backend validates and saves them in PostgreSQL; the student dashboard retrieves the resulting attendance.

Deliverables: source code, database scripts and migrations, API implementation, frontend components, unit tests, build artifacts, and developer documentation.

Interview: Coding follows requirements analysis and design so developers know what to build and how components should interact. Starting too early increases rework and the risk of missing functionality.

6. Phase 5 — Testing

Software testing evaluates software to identify defects and verify that it satisfies specified requirements. For SmartAttend, testing checks attendance accuracy, permissions, reports, and the end-to-end workflow.

Verification vs. validation

Verification	Validation
Are we building the product right?	Are we building the right product?
Reviews whether work conforms to specifications.	Checks whether the product meets user needs and intended use.
Examples: design review, code review, checking constraints.	Examples: teachers try the workflow, administrators approve reports.

Testing levels

Level	Purpose	SmartAttend example
Unit	Test a component in isolation.	Check the attendance calculator returns 90% for 36/40.
Integration	Test interactions between components.	Submit attendance through the API and verify database persistence.
System	Test the complete application.	Log in as teacher, mark attendance, then verify as student.
Acceptance	Check agreed acceptance criteria.	Teachers and administrators approve the workflow and reports.

Attendance calculator test cases

Input	Expected result
36 attended / 40 total	90%
20 attended / 40 total	50%
0 attended / 40 total	0%
40 attended / 40 total	100%
0 attended / 0 total	0% under the example policy
-1 attended / 40 total	Validation error
41 attended / 40 total	Validation error

Functional and non-functional testing

Type	Examples
Functional	Login, saving attendance, percentage calculation, report generation.
Performance	Measure response time under expected workload.
Load	Check behavior with many simultaneous users.
Stress	Examine behavior beyond normal operating conditions.
Security	Test authentication, authorization, and common vulnerabilities.
Usability	Check whether teachers can use the interface easily.
Compatibility	Verify supported browsers and devices.
Recovery	Check recovery after failures.

Regression, smoke, and sanity testing

Test type	Meaning	Example
Regression	Check old functionality after changes.	After adding reports, rerun login and attendance tests.
Smoke	Check basic stability of a new build.	Application starts, login page loads, database connection works.
Sanity	Focused checks after a limited change or fix.	After fixing a formula, verify that calculation and nearby behavior.

Black-box, white-box, and gray-box testing

Black-box: evaluate inputs and outputs without relying on internal code knowledge. **White-box:** examine internal logic and code paths. **Gray-box:** combine behavioral testing with some knowledge of internal implementation.

Defect life cycle



Example defect: a tester reports that 36 out of 40 classes displays 80% instead of 90%. The report should include a defect ID, description, reproduction steps, expected and actual results, severity, and priority. The developer fixes it and the tester retests.

Severity vs. priority

Severity describes the impact of a defect. Priority describes how urgently it should be fixed. A vulnerability that lets students modify their own attendance may be high severity and high priority; a minor alignment issue on a rarely used screen may be low in both.

> **Deliverables: test plan, test cases, test data, execution reports, defect reports, regression results, acceptance results, and release recommendation.**

Interview: Unit testing checks individual components in isolation; integration testing checks whether components interact correctly.

7. Phase 6 — Deployment

Deployment makes software available in its intended environment. For SmartAttend, it means setting up the application so authorized students, teachers, and administrators can use it.

Deployment activities

- Prepare the production environment and configure the database.
- Apply database migrations and configure settings and secrets.
- Build and package the application.
- Deploy the frontend and backend.
- Configure HTTPS, access controls, and monitoring.
- Run smoke tests, prepare user guidance, and verify recovery procedures.

Environments

Environment	Purpose
Development	Developers write and test code.
Testing / QA	Testers validate features and integrations.
Staging	Rehearse release procedures in a production-like environment.
Production	Real users perform their actual work.

Deployment strategies

Strategy	SmartAttend example	Trade-off
Direct / big-bang	College switches on the announced launch date.	Simple transition but higher cutover risk.
Phased	Release department by department.	Manageable rollout, but coordination is needed.
Pilot	One department tests for two weeks before expansion.	Finds issues early, but takes extra time.
Parallel	Use paper registers and SmartAttend together temporarily.	Fallback and comparison, but more work.

Rollback plan

A rollback plan defines how to return to a previous stable release or recover safely if deployment causes serious problems. It may include keeping the previous application version, backing up before risky migrations, testing recovery, and defining criteria for reversing a release. Database changes need special care: reverting application code alone may not safely reverse data changes.

Deliverables: production application, release notes, deployment records, user documentation, monitoring, backup, and rollback procedures.

Interview: Staging validates a release in a production-like environment; production is the live environment used by real users.

8. Phase 7 — Maintenance

Maintenance modifies software after delivery to correct defects, adapt to environmental changes, improve quality, or add approved capabilities. Development does not end at deployment.

Four types of maintenance

Type	Purpose	SmartAttend example
Corrective	Fix defects.	Correct a wrong attendance percentage.
Adaptive	Adapt to environmental changes.	Update database connectivity after an infrastructure upgrade.
Perfective	Improve functionality or quality.	Add downloadable attendance-shortage reports.
Preventive	Reduce future maintenance risk.	Refactor complex code, remove obsolete dependencies, add regression tests.

Memory trick: Corrective = correct defects; Adaptive = adapt to changes; Perfective = improve; Preventive = reduce future problems.

Maintenance activities

- Bug fixing and security patching.
- Performance optimization and dependency upgrades.
- Database maintenance and backup verification.
- Monitoring and incident investigation.
- Documentation updates and feature enhancements.

Deliverables: updated releases, bug fixes, security updates, improved documentation, updated tests, and maintenance records.

9. Complete SDLC Walkthrough — SmartAttend

Scenario: a college with 2,000 students and 100 teachers wants to replace manual attendance registers. The numbers are illustrative and used only to make the example concrete.

Phase	What the team does	Example	Output
Planning	Define problem, scope, feasibility.	Determine online attendance is practical.	Project plan
Requirements	Collect and validate needs.	Specify login, marking, and reports.	SRS
Design	Define architecture, data, UI, APIs.	Design React + Spring Boot + PostgreSQL.	Design documents
Implementation	Write and integrate code.	Build login, attendance, reports.	Working application
Testing	Verify requirements and find defects.	Check calculations, permissions, workflows.	Test reports
Deployment	Release to users.	Launch for the college.	Production release
Maintenance	Fix, adapt, and improve.	Add reports and security updates.	Updated releases

What if we skip a phase?

Skipped phase	Possible consequence
Planning	Unnecessary features, unrealistic budget, unclear goals.
Requirements	Missing features or incorrect attendance rules.
Design	Poor architecture or inconsistent database structure.
Implementation standards	Code becomes hard to understand, test, and maintain.
Testing	Incorrect percentages or unauthorized access reach production.
Deployment planning	Release fails because of configuration or migration problems.
Maintenance	Defects and security issues remain unresolved.

Key lesson: each phase reduces particular risks and prepares the project for the next stage. In iterative approaches, phases may overlap and earlier decisions may be revisited.

10. SDLC Models

An SDLC model defines how development activities are organized. A stable, well-understood project may suit a sequential model; a project with changing requirements may benefit from iterative development.

10.1 Waterfall Model

Definition: A sequential approach in which phases are generally completed before the next begins.

SmartAttend example: SmartAttend: finalize requirements, approve design, implement, test, then deploy. Changes are handled through a controlled process.

Advantages: Simple to understand; clear milestones; easier to plan when requirements are stable.

Disadvantages: Changes can be expensive later; working software may arrive late; less suitable for frequently changing requirements.

Best suited for: Stable requirements, familiar technology, formal documentation.

10.2 V-Model

Definition: Pairs development stages with corresponding verification and validation activities; test planning begins early.

SmartAttend example: SmartAttend: define acceptance tests while requirements are written, integration tests while APIs are designed, and unit tests while component logic is designed.

Advantages: Early test planning; traceability; clear quality activities.

Disadvantages: Less flexible when requirements change; changes can affect many test artifacts.

Best suited for: Stable requirements and projects with strong quality or compliance needs.

10.3 Iterative Model

Definition: Develops a solution through repeated cycles that refine and improve it.

SmartAttend example: SmartAttend: first build login, then improve validation, then add attendance, then refine reports.

Advantages: Feedback and learning; early discovery of problems.

Disadvantages: May require rework; repeated cycles need coordination.

Best suited for: Requirements become clearer during development.

10.4 Incremental Model

Definition: Delivers software in functional parts, with each increment adding capabilities.

SmartAttend example: SmartAttend: increment 1 login; increment 2 attendance; increment 3 percentage calculation; increment 4 reports.

Advantages: Useful functionality earlier; visible progress; prioritization.

Disadvantages: Requires architectural planning; integration and dependencies can complicate delivery.

Best suited for: Staged delivery and early value.

10.5 Prototyping Model

Definition: Uses an early sample or limited working version to clarify requirements and gather feedback.

SmartAttend example: SmartAttend: show teachers a mock attendance screen, collect feedback, then refine requirements and design.

Advantages: Clarifies ambiguity; improves communication and usability.

Disadvantages: Users may mistake prototype for finished software; scope can expand.

Best suited for: Users cannot fully express needs until they see a sample.

10.6 Spiral Model

Definition: A risk-driven approach using repeated cycles of objectives, risk analysis, engineering, and evaluation.

SmartAttend example: SmartAttend: investigate identity integration, load, data migration, and recovery risks early, then build and evaluate successive versions.

Advantages: Strong risk focus; supports refinement; useful for complex projects.

Disadvantages: Can be costly and requires risk-analysis expertise.

Best suited for: Large, complex, high-risk projects.

10.7 RAD Model

Definition: Rapid Application Development emphasizes fast prototyping, reusable components, and frequent user feedback.

SmartAttend example: SmartAttend: use existing UI libraries, reusable login components, and rapid prototypes to deliver a basic version quickly.

Advantages: Can shorten delivery; frequent feedback; reuse.

Disadvantages: Needs skilled developers and available stakeholders; speed does not remove testing.

Best suited for: Time-sensitive, modular applications with frequent user feedback.

10.8 Agile Model

Definition: An adaptive approach emphasizing collaboration, frequent delivery, customer feedback, and response to change.

SmartAttend example: SmartAttend: two-week sprints deliver login, attendance, dashboard, then reports, with reviews after each sprint.

Advantages: Adapts to change; working software arrives frequently; visible progress.

Disadvantages: Needs stakeholder participation and disciplined backlog management.

Best suited for: Evolving requirements and frequent feedback.

10.9 Big Bang Model

Definition: Development proceeds with relatively little formal planning; the solution evolves as developers work.

SmartAttend example: SmartAttend: a student makes a small attendance calculator experiment. This would be risky for a college-wide system with sensitive data.

Advantages: Low initial process overhead; useful for small experiments.

Disadvantages: High uncertainty, difficult estimates, and limited formal risk control.

Best suited for: Small experiments and learning projects with limited consequences.

Model comparison

Model	Main characteristic	Flexibility	Best suited for
Waterfall	Sequential phases	Low–moderate	Stable requirements
V-Model	Development paired with testing	Low–moderate	Quality-critical work
Iterative	Repeated refinement	High	Evolving understanding
Incremental	Functional parts delivered over time	High	Early staged delivery
Prototyping	Early samples and feedback	High during clarification	Unclear requirements
Spiral	Risk-driven cycles	High	Complex, high-risk projects
RAD	Rapid prototyping and reuse	Moderate–high	Time-sensitive modular projects
Agile	Frequent incremental delivery	High	Changing requirements
Big Bang	Minimal formal structure	Unpredictable	Small experiments

Waterfall vs. Agile

Waterfall	Agile
Primarily sequential.	Iterative and incremental.
Detailed requirements are usually established early.	Priorities can evolve with feedback.
Late changes may be expensive.	Designed to accommodate change.
Delivery often follows completion of major phases.	Working software is delivered frequently.
Formal phase gates may be emphasized.	Collaboration and feedback are emphasized.

11. Agile, Scrum, and DevOps

Agile is a set of values and principles for adaptive, collaborative software development. Scrum is a framework that helps teams organize iterative and incremental work. Scrum is one way to practice Agile, but Agile is broader than Scrum.

Scrum accountabilities

Accountability	Responsibility
Product Owner	Maximizes product value and manages the Product Backlog.
Scrum Master	Helps establish Scrum and improve team effectiveness.
Developers	Create a usable Increment during each Sprint.

Scrum artifacts: Product Backlog, Sprint Backlog, and Increment. Scrum events: Sprint, Sprint Planning, Daily Scrum, Sprint Review, and Sprint Retrospective.

SmartAttend Scrum example

- The Product Owner prioritizes attendance marking above optional visual effects.
- Developers implement and test attendance marking during the sprint.
- Teachers evaluate the feature during the Sprint Review.
- The team identifies process improvements during the Sprint Retrospective.

What is DevOps?

DevOps is a set of practices and cultural principles that improve collaboration between development and operations. It emphasizes automation, continuous feedback, reliable releases, and monitoring.

```
Plan → Code → Build → Test → Release → Deploy → Operate → Monitor
  ^                                                        |
  +------------------------ Feedback --------------------------+
```

CI vs. CD

Term	Meaning
Continuous Integration (CI)	Frequently integrate code changes and automatically build and test them.
Continuous Delivery	Keep software release-ready; production release often requires approval or an explicit decision.
Continuous Deployment	Automatically release qualifying changes to production after required checks pass.

SmartAttend pipeline example: a developer changes attendance logic. The pipeline builds the Java application, runs unit tests and quality checks, and reports failures. If checks pass, the release pipeline prepares it for deployment; production deployment may be automatic or require approval.

12. Related Concepts and Documentation

SDLC vs. STLC

SDLC	STLC
Covers the broader software development process.	Focuses on structured testing activities.
Includes planning, requirements, design, implementation, testing, deployment, and maintenance.	Includes test planning, analysis, design, environment preparation, execution, and closure.
Involves the wider project team.	Primarily focuses on testing responsibilities, with cross-team collaboration.

STLC operates within or alongside SDLC; it does not replace it. Agile is an approach that can organize both development and testing into frequent iterations.

Alpha testing vs. beta testing

Alpha testing: typically performed internally or in a controlled setting before wider external evaluation. **Beta testing:** gives a pre-release version to selected real users in a more realistic environment. For SmartAttend, staff might test internally first, then a selected group of teachers could try a pilot version.

Error vs. defect vs. failure

Error: a human mistake, such as misunderstanding a formula. **Defect:** a flaw in software, such as an incorrect formula in code. **Failure:** observable incorrect behavior at runtime, such as displaying the wrong attendance percentage.

Milestones and Gantt charts

A milestone is a significant point in the schedule, such as SRS approval, design approval, completion of core features, testing completion, or deployment. A Gantt chart shows tasks and their scheduled durations over time.

Activity	Weeks 1–2	Weeks 3–4	Weeks 5–6	Weeks 7–9	Weeks 10–11	Week 12
Planning	✓					
Requirements	✓	✓				
Design		✓				
Implementation			✓	✓		
Testing				✓	✓	
Deployment						✓

This schedule is illustrative. Real work may overlap, and teams should reserve time for integration, defect correction, and release preparation.

Deliverables

Phase	Example deliverable
Planning	Project plan
Requirements	SRS
Design	Architecture and database design
Implementation	Source code
Testing	Test report
Deployment	Production release
Maintenance	Updated release and maintenance records

Requirement Traceability Matrix (RTM)

An RTM links requirements to design elements, implementation components, and test cases. It helps show that each requirement is implemented and verified and helps identify the impact of requirement changes.

Requirement ID	Requirement	Implementation	Test case
FR-01	User login	Authentication service	TC-01
FR-03	Mark attendance	Attendance API	TC-05
FR-05	Calculate percentage	Attendance calculator	TC-09
FR-07	Generate reports	Reporting service	TC-12

13. Common SDLC Problems and Best Practices

Problem	Why it matters	Useful response
Unclear requirements	Wrong or missing functionality.	Interviews, observation, prototypes, SRS validation.
Scope creep	Unplanned work increases cost and delays.	Change control and prioritization.
Poor design	Architecture becomes difficult to maintain.	Review architecture and keep components modular.
Insufficient testing	Defects reach production.	Automated tests, code reviews, end-to-end tests.
Security weaknesses	Sensitive data or access may be exposed.	Secure design, backend authorization, security testing.
Schedule overruns	Tasks exceed estimates.	Track dependencies, uncertainty, and milestones.
Poor maintenance	Software becomes harder to change.	Refactoring, documentation, dependency updates, regression tests.

Best-practice checklist

- Involve stakeholders early and often.
- Document and validate requirements.
- Choose a model suited to project constraints and risks.
- Design security into the system.
- Use version control and code reviews.
- Automate appropriate tests.
- Test integrations and critical user workflows.
- Maintain clear release and rollback procedures.
- Monitor production behavior and collect user feedback.
- Keep documentation and regression tests up to date.

14. SDLC Interview Questions and Answers

1. What is SDLC?

A systematic process for planning, designing, developing, testing, deploying, and maintaining software.

2. What are the phases of SDLC?

A common seven-phase structure is planning/feasibility, requirements analysis, design, implementation, testing, deployment, and maintenance.

3. Why is SDLC important?

It structures requirements, cost, schedule, risks, quality, delivery, and maintenance.

4. What is usually the first phase?

Planning is commonly treated as the first phase.

5. Which phase produces the SRS?

Requirements analysis.

6. Which phase converts design into source code?

Implementation.

7. Which phase identifies defects?

Testing is the primary systematic defect-finding phase, though reviews and other activities can also find defects.

8. Does SDLC end after deployment?

No. Maintenance continues after deployment.

9. What is a feasibility study?

An evaluation of technical, economic, legal, operational, and schedule practicality.

10. What is an SRS?

A structured document specifying functional and non-functional requirements, constraints, interfaces, and other agreed requirements.

11. Functional vs. non-functional requirements?

Functional requirements specify what the system does; non-functional requirements specify quality attributes and constraints.

12. HLD vs. LLD?

HLD describes overall architecture; LLD describes component internals such as classes, methods, algorithms, and constraints.

13. Verification vs. validation?

Verification checks conformance to specifications; validation checks whether user needs and intended use are met.

14. What is regression testing?

Checking that changes have not broken previously working functionality.

15. Smoke vs. sanity testing?

Smoke testing checks basic build stability; sanity testing focuses on a specific change or fix.

16. What is a prototype?

An early representation used to explore, clarify, or validate requirements and design.

17. Which SDLC model is best?

There is no universal best model; select based on requirements, risks, constraints, and feedback needs.

18. Can Agile and SDLC be used together?

Yes. SDLC describes the broader life cycle; Agile organizes development adaptively.

19. Iterative vs. incremental development?

Iterative development refines a solution; incremental development adds functional parts. A project can use both.

20. What is technical debt?

Future cost created by shortcuts or design decisions that make later changes harder.

21. What is a rollback plan?

A plan for returning to a stable release or recovering safely when deployment fails.

22. What is risk management?

Identifying, evaluating, prioritizing, and addressing uncertainties affecting project objectives.

23. What is a CI/CD pipeline?

Automation for building, testing, packaging, and releasing software.

24. Why is documentation important?

It communicates requirements, design, interfaces, testing evidence, deployment procedures, and maintenance information.

25. Defect vs. failure?

A defect is a flaw in software or another artifact; a failure is observable incorrect behavior at runtime.

15. Practice MCQs

1. Which document describes functional and non-functional requirements?

A. Test report

B. SRS

C. Deployment log

D. Source code

Answer: B — SRS

2. Which phase converts approved design into source code?

A. Planning

B. Requirements analysis

C. Implementation

D. Maintenance

Answer: C — Implementation

3. Which model explicitly emphasizes risk analysis in repeated cycles?

A. Waterfall

B. Spiral

C. Big Bang

D. V-Model

Answer: B — Spiral

4. Checking whether a teacher can save attendance is an example of:

A. Functional testing

B. Economic feasibility

C. Preventive maintenance

D. System design

Answer: A — Functional testing

5. Which maintenance adapts software to a changed environment?

A. Corrective

B. Adaptive

C. Perfective

D. Preventive

Answer: B — Adaptive

6. Which approach emphasizes frequent delivery and feedback?

A. Waterfall

B. Big Bang

C. Agile

D. V-Model

Answer: C — Agile

7. Which test level verifies frontend/backend/database interaction?

A. Unit

B. Integration

C. Acceptance

D. Usability

Answer: B — Integration

8. Which technique helps clarify unclear requirements with an early sample?

A. Prototyping

B. Rollback

C. Deployment

D. Regression testing

Answer: A — Prototyping

9. What does verification primarily check?

A. User preference

B. Conformance to specifications

C. Profitability

D. Color choice

Answer: B — Conformance to specifications

10. What is the purpose of an RTM?

A. Replace source code

B. Link requirements to implementation and tests

C. Calculate budget

D. Deploy application

Answer: B — Link requirements to implementation and tests

Final Revision — The Entire SDLC in Five Minutes

Phase	Key question	Remember	Deliverable
1. Planning	Why are we building it?	Problem, scope, feasibility, budget, risks.	Project plan
2. Requirements	What must it do?	Gather, analyze, validate needs; write SRS.	Approved requirements
3. Design	How will we build it?	Architecture, database, APIs, UI, security.	Design documents
4. Implementation	Build it.	Code, integrate, review, version control.	Working software
5. Testing	Does it work correctly?	Verify requirements, detect defects, retest.	Test results
6. Deployment	Make it available.	Configure, release, smoke-test, monitor.	Production release
7. Maintenance	Keep it useful.	Correct, adapt, improve, prevent future issues.	Updated releases

Most important takeaway

SDLC is not merely a sequence of coding activities. It is a systematic way to manage requirements, design, implementation, quality, release, and long-term software evolution. SDLC models such as Waterfall, V-Model, Spiral, and Agile describe different ways to organize that work.

Study tip: First memorize the seven phases and their deliverables. Then compare the SDLC models and testing concepts. Finally, explain the complete SmartAttend project in your own words without looking at the notes.